The CNN lab: a convolutional network on CIFAR-10, same loop

AI Methods for Science (77222) · Part I, week 2; continues `lab_mlp.pdf`

The setup

`cnn_lab.py` (from https://github.com/yohai/aims-lab-session2; next to `mlp_lab2.py`, which it imports) has the same `train(config)` interface and the same result format (plus `result["model"]`, the trained network). Only the ansatz changed: a stack of blocks (conv $3 \times 3 \to$ BatchNorm $\to$ ReLU, then 2×2 max-pooling), global average pooling, one linear layer. The defaults are Monday's recipe applied to a small CNN. The new keys:

key	default	meaning
`channels`	`[32, 64, 128]`	output channels of each block; `[]` gives the MLP of `lab_mlp.pdf` (`hidden [512, 256]`) inside the same script
`convs_per_block`	`1`	2 puts two conv layers in every block (VGG-style)
`kernel`	`3`	size of every conv kernel
`batchnorm`	`true`	BatchNorm after every conv
`shuffle_pixels`	`false`	one fixed random permutation of the 3072 pixels, applied to every image, training and validation alike
`skip`	`false`	a skip connection around every conv: ResNet's residual units (E22)
`classes`	`[]`	which of the ten classes to use, e.g. `[0,1,2,3,4]`; labels renumbered $0..K-1$ (E20)
`n_per_class`	`0`	keep only this many training images per class; `0` = all (E20)
`save_weights`	`""`	a path; the weights of the best epoch are saved there after training (E20)
`init_from`, `freeze`	`""`, `false`	start the conv blocks from saved weights, with a new linear head; `freeze` trains the head only (E20)
`optimizer`, `lr`, `weight_decay`, `schedule`, `augment`, ...	AdamW, 10^{-3}, 0.05, cosine, true	as in `mlp_lab2.py`; defaults are the E15 recipe
`epochs`	`10`	
`device`	`"auto"`	GPU (`cuda`) or Apple GPU (`mps`) if present, else CPU

Compute

A conv net costs more per epoch than the MLP. Default network, 352 steps: ≈ 6 s per epoch on an Apple M-series GPU, ≈ 20 s on an M5 CPU, 40–60 s on an older laptop CPU. **On a CPU, run 5 epochs instead of 10 and say so.** Do E16 first; if time is short, skip to E18 and do E17 at home. E20 starts in class on Thursday; E19, E20, E21 and E22 are the homework after it (`eurosat_lab.py`; run `python eurosat_lab.py -download` at home first). Rules as before: do not edit the scripts; predict before each run; the test set once, in E18.

The report

E16–E22 go into the same kind of report as E0–E15 (see `lab_mlp.pdf`): prediction, table, figures, a few sentences each, one PDF per group, submitted as a GitHub issue on the lab repository: https://github.com/yohai/aims-lab-session2/issues/new/choose, *Lab report*. Which experiments are done in class and which at home is announced in class. Our reference curves are shown next to yours.

The experiments

E16. A CNN against the MLP, and what it is that it knows

~4 min on a GPU

Run. 10 epochs each: (a) the default CNN; (b) `channels = []`, the MLP, under the same recipe; (c) the CNN with `shuffle_pixels = true`; (d) the MLP with `shuffle_pixels = true`.

Collect. validation accuracy vs. epoch, four runs on one axis; a table with `n_params`, `val_acc_at_best`, `sec_per_epoch`.

Compute. Four runs of 10 epochs: ≈ 4 min on an Apple GPU; on an M5 CPU the two CNN runs cost ≈ 20 s per epoch, the two MLP runs seconds, so 5 epochs each, ≈ 4 min in all.

Answer. Before running: which of the four will be hurt by the shuffle, and by how much? Afterwards: the CNN has 18× fewer parameters than the MLP. What does it know that the MLP does not? Why is the MLP unhurt by the shuffle? After the shuffle, is the CNN still better than the MLP, and why could it be?

E17. Width and depth of the CNN

~5 min on a GPU

Run. 10 epochs: `channels` ∈ {`[8,16,32]`, `[16,32,64]`, `[32,64,128]` (E16a), `[64,128,256]`}; if you have a GPU, also `[64,128,256]` with `convs_per_block = 2`.

Collect. `val_acc_at_best` vs. `n_params` (log x), with E16's MLP as one more point; `sec_per_epoch` in the table.

Compute. The cost grows with the channels: `[8,16,32]` and `[16,32,64]` are cheaper than the default; `[64,128,256]` is ≈ 4× the default (10 s per epoch on an Apple GPU, ≈ 80 s on an M5 CPU); the two-convs version is ≈ 12× (25 s per epoch on the GPU, minutes on a CPU): GPU only. On a CPU: the three small configurations at 5 epochs, `[64,128,256]` at 3.

Answer. How many parameters does a CNN need to beat the 1.7M-parameter MLP? Is the gain per parameter still rising at the top of your range? Where is the time going as the channels double (count the multiply-adds of one conv layer: $k^2\, C_{\text{in}}\, C_{\text{out}}\, H\, W$)?

E18. Your best CNN, test once

30 epochs; at home on a CPU

Run. the best architecture of E17 for 30 epochs. Choose by validation; then one run with `test = true`.

Collect. training and validation accuracy and validation loss vs. epoch; the table row; the test number.

Compute. 30 epochs of the default net: ≈ 3 min on an Apple GPU, ≈ 10 min on an M5 CPU, longer on an older laptop; of `[64,128,256]`: 5 min and 40 min. The two-convs version at 30 epochs is a GPU-only run (≈ 12 min). Start it, go for lunch.

Answer. Where does the validation loss bottom out now, and does early stopping still matter with cosine + augmentation? How far is your CNN from the best MLP of E15? Humans are at ≈ 94% on CIFAR-10 and the best published networks above 99%: name two things you would change next.

E19. Take things away (at home)

~4 min on a GPU

Run. 10 epochs, from the default: `batchnorm = false`; `augment = false`; `optimizer = "sgd"` with `momentum` 0.9, `lr` = 0.05, `weight_decay` = 5×10^{-4} (the classic CNN recipe).

Collect. training and validation accuracy vs. epoch, the three and the default on one axis.

Compute. Three runs of 10 epochs at the default cost: ≈ 3 min on a GPU, ≈ 10 min on an M5 CPU at 10 epochs, half at 5.

Answer. Which removal costs the most? Without augmentation, does the CNN overfit the way the

MLP did in E4 (training 100%, validation loss rising)? Without BatchNorm, what happens in the first epochs, and does it recover? Does the classic SGD recipe match AdamW at 10 epochs?

E20. What does a trained network know about classes it never saw? ~1 min on a GPU, 5 on a CPU

Run. (a) pretrain: `classes = [0,1,2,3,4]` (plane, car, bird, cat, deer), 10 epochs, `save_weights = "pre.pt"`. Then the other five classes, B = `[5,6,7,8,9]` (dog, frog, horse, ship, truck), with `n_per_class` $\in \{10, 100, 1000\}$, four ways: (b) **no training**: `nearest_mean("pre.pt", B, n)`, which pushes the images through the frozen conv blocks, takes the mean feature vector of each class over its n labelled images, and assigns each validation image to the nearest mean; (c) **head only**: `init_from = "pre.pt"`, `freeze = true`, and the head's own recipe, `lr` = 0.01, `batch_size` = 32, `epochs` = 100, `augment = false`; (d) **fine-tune**: `init_from = "pre.pt"`, no freeze, 10 epochs; (e) **from scratch**, 10 epochs.

Collect. validation accuracy vs. the number of labelled images (50, 500, 5000; log x), four curves on one axis; `n_params` (the trainable ones) in the table.

Compute. Pretraining on five classes: ≈ 30 s on a GPU, ≈ 2 min on an M5 CPU (half the images). Everything else is on 50–5,000 images: seconds each, except the 100-epoch head runs (≈ 15 s each). Nearest mean: seconds. Whole experiment ≈ 1 min on a GPU, ≈ 5 min on a CPU.

Answer. Before running: with 50 images, rank the four, and say by how much (b) beats or loses to (e). Afterwards: the pretraining never saw a dog or a ship, and (b) trains nothing; what did the conv blocks learn that works on dogs? Why does the frozen head need a different learning rate and many more epochs than the default (count its gradient steps under the default)? Where do the curves cross, and what would you do with 50 labelled images of your own?

E21: transfer learning on EuroSAT (homework)

EuroSAT (Helber et al. 2019): 27,000 Sentinel-2 satellite patches, 64×64 RGB, 10 land-use classes (crop, forest, highway, industrial, pasture, residential, river, sea/lake, ...). `eurosat_lab.py` loads it with a fixed split, 5,000 test, 2,000 validation, and `n` labelled training images drawn from the rest, $n/10$ per class; `train(config)` has the same interface and result format as before. Run `python eurosat_lab.py -download` once, at home, before you start (140 MB). Two ways to do the experiment; pick one, or do the first and then the second.

Option A: the four methods by key	**Option B, the challenge: build the models yourself**
`method` ∈ {`"mlp"`, `"scratch"`, `"frozen"`, `"finetune"`}: the session-2 MLP on the pixels; ResNet-18 from random initialization; the ImageNet-pretrained ResNet-18 as a frozen feature extractor with a new linear head; the same with every weight trained at a small learning rate. `epochs`, `lr`, `batch_size` default to the method's own (0 = default: 40 / 30 / 100 / 15 epochs).	Pass your own network: `train(config, model=m)`; any `nn.Module` from a $(B, 3, \texttt{size}, \texttt{size})$ batch to 10 logits. Build the four yourself: the MLP; `resnet18(num_classes=10)`; the pretrained one, `resnet18(weights=ResNet18_Weights.IMAGENET1K_V1)` with `m.fc = nn.Linear(512, 10)`, fine-tuned; and the same frozen. For the frozen one, do not run the backbone every epoch: set `backbone.fc = nn.Identity()`, compute `feats = extract_features(backbone)` once ($27{,}000 \times 512$; ≈ 8 min on a CPU, cache it with `torch.save`), then `train(config, model=nn.Linear(512, 10), feats=feats)`. Details that bite: the input is already ImageNet-standardized and resized to `size`; the pretrained net wants `size` = 128; a frozen backbone belongs in `eval()` mode; fine-tune with `lr` ≈ 10^{-4}, the head alone with ≈ 10^{-2} and many epochs.

Compute (Apple M5 CPU; an older laptop CPU is 2–3× slower)

`frozen`: the features are computed once for every image used and cached, ≈ 40 s for $n = 300$ with the test set, then the head trains in seconds. `mlp`: seconds. `finetune` and `scratch`: ≈ 16 s per epoch at $n = 300$ (most of it the validation pass), ≈ 30 s at $n = 1000$; so 15 epochs of fine-tuning take 4–8 minutes and 30 epochs from scratch 8–15. On a GPU all of it is minutes. If short of time: all four methods at $n = 300$, then `frozen` at every n.

E21. Transfer learning on EuroSAT (homework)

~30 min on a CPU

Run. `n` ∈ {100, 300, 1000}, the four methods (option A or B), the method's default epochs; `test` = `true` on every run (each run is one experiment; the test set is not used to choose anything here).

Collect. test accuracy vs. n (log x), four curves on one axis; a table with `n_params` (the trainable ones), `best_epoch`, `sec_per_epoch`; for $n = 1000$, the validation curves of the four methods.

Answer. Before running: at $n = 100$ (10 images per class), which method wins, and by how much? At $n = 1000$? Afterwards: the frozen backbone never saw a satellite image and only 5,130 numbers were trained; what did ImageNet teach it that transfers? Does fine-tuning beat the frozen features at every n, and where does the gap open? Would you expect the same on data that are not photographs (spectra, time series)? One paragraph: how you would classify a new set of 200 labelled microscope images tomorrow.

E22. Depth with and without skips (homework)

~7 min on a GPU; 5 epochs on a CPU

Run. `convs_per_block` = 4 (twelve conv layers), `batchnorm` = `false`, 10 epochs: once as is, once with `skip` = `true` (the identity added around every conv, a 1×1 projection where the channel count changes). With a GPU: both again with `batchnorm` = `true`.

Collect. training loss vs. epoch, the runs on one axis.

Compute. 4× the default cost per epoch (14–29 s on an Apple GPU, ≈ 80 s on an M5 CPU): ≈ 3 min per run on a GPU, ≈ 7 min at 5 epochs on a CPU. The two BatchNorm-on runs are the same again:

GPU only.

Answer. Before running: sketch the two training curves; by how much will the skips help, if at all, at twelve layers? Afterwards: the gap in training loss at epoch 10, and in validation accuracy; is it a speed difference or a different endpoint? With BatchNorm on, how much of the gap closes? What do the skips change that BatchNorm does not (the week-2 depth frame: a product of factors vs. a sum)?